

01/10) 3. Cookie & Session (1)

서론

HTTP 헤더에 들어가는 클라이언트의 인증 정보로 서버 입장에서 다른 클라이언트를 구별할 수 있음.

쿠키

- IP 주소와 User-Agent는 고유성이 없음
- HTTP 프로토콜의 특징(Connectionless, Stateless)

위 두 가지 문제로 웹 서버에서 클라이언트를 기억할 수 없음.

이때 HTTP에서 클라이언트의 상태를 유지하기 위해 생긴 것이 **쿠키(Cookie)**

쿠키란

쿠키는 키(Key)와 값(Value)로 이루어진 단위로, 서버가 클라이언트에게 발급하면 클라이언트에 서버에 요청을 보낼 때 쿠키를 함께 보내 서버에서 구별할 수 있게 함.

용도

클라이언트의 정보기록 및 상태정보 저장

- 정보기록: 팝업 옵션 등 클라이언트별 정보를 기록. 쓸데없는 쿠키 전송을 막기 위해 요즘에는 Modern Storage APIs를 사용함
- 상태정보: 클라이언트별 사용자를 식별하고 로그인 상태를 기억하기 위한 정보를 쿠키에 기록함

쿠키 변조

쿠키 발급은 서버가 하지만 저장은 클라이언트의 브라우저가 하므로, 서버에서 쿠키에 대해 검증하지 않으면 공격자가 쿠키 정보를 변조해 서버에 요청을 보낼 위험성이 있음.

HTTP 프로토콜 특징

Connectionless : 하나의 요청에 하나의 응답을 한 후 연결 종료하는 것. 요청끼리 이어지지 않고 요청이 발생할 때마다 새로운 연결 생성됨

Stateless : 통신이 끝난 후 상태를 저장하지 않음. 따라서 이전 연결의 정보를 이후 연결에 요구할 수 없음

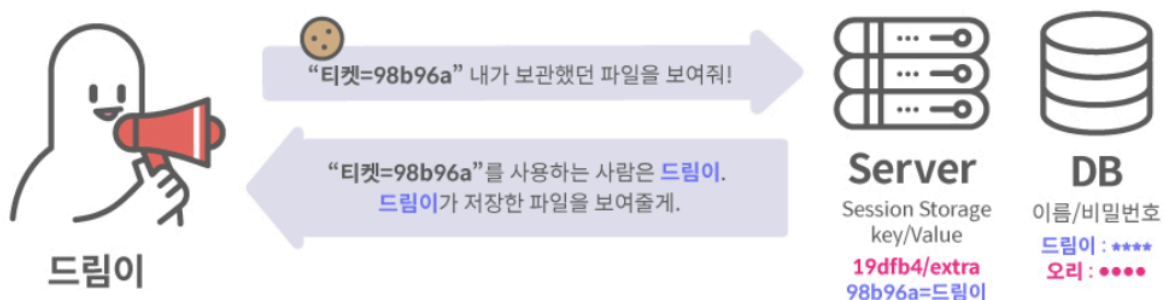
세션

클라이언트가 인증 정보를 변조할 수 없게 하기 위해 사용함

인증 정보를 서버에 저장하고, 정보에 접근할 수 있는 **Session ID**(랜덤한 문자열)를 제공.

브라우저에서 이것을 쿠키에 저장하고 서버 요청마다 보냄

쿠키는 데이터를 클라이언트가 저장하는 반면, 세션은 서버에 저장



쿠키 적용법

클라이언트에 저장되므로 이용자가 쿠키를 조회, 수정, 추가 가능

쿠키 설정 시 만료 시간 지정 가능, 시간 지나면 클라이언트에서 쿠키 삭제됨

서버에서 쿠키 설정하는 법

- HTTP 응답에서 쿠키 설정 헤더(Set-Cookie) 추가

```
HTTP/1.1 200 OK
Server: Apache/2.4.29 (Ubuntu)
Set-Cookie: name=test;
Set-Cookie: age=30; Expires=Fri, 30 Sep 2022 14:54:50 GMT;
...
```

클라이언트에서 쿠키 설정하는 법

- 자바스크립트 사용

```
document.cookie = "name=test;"
document.cookie = "age=30; Expires=Fri, 30 Sep 2022 14:54:50 GMT;"
```

콘솔에서 쿠키 정보 조회

개발자도구 → console 에서 `document.cookie` 입력

```
> document.cookie
< '118n_redirected=ko; _ga=GA1.1.1126870330.1700056463; csrf_token=htG0NmCsHEesbwV0gCB81U9FpR6VQn8jI0qZenR0haHq3qjDUKfbq54eagoZC9; _click=w0yuvz%7C2%7CF19%7C0%7C1414; _clsk=k2p1gy%7C1704778450370%7C1%7C1%7Ccy.clarity.ms%2Fcollect; roadmap_recent=%7B%22id%22%3A1%2C%22title%22%3A%22Web%20Hacking%22%7D; objective_recent=%7B%22id%22%3A4%2C%22title%22%3A%22Cookie%20%26%20Session%22%7D; _ga_GYX8VR6KG3-GS1.1.1704778454.23.1.1704778788.44.0.0'
```

Application에서 조회

개발자도구 → Application → Cookies

세션 연습

1. Network에서 Preserve log체크 후 로그인하여 응답 보기
2. set-cookie 에서 세션 아이디 확인
3. application에서 sessionID 지우면 로그인이 풀림
4. 다시 추가하면 로그인 됨

Name	Headers	Preview	Response	Initiator	Timing	Cookies
75rr7kaoas	General					
2ffa51e.css	Request URL:	https://dreamhack.io/api/v2/activity/timeline/46265/				
0bfaef4.js	Request Method:	GET				
collect?v=2&tid=G-GY...	Status Code:	200 OK				
follow_count/?user_id=...	Remote Address:	43.202.106.113:443				
46265/	Referrer Policy:	strict-origin-when-cross-origin				
communities/?page=1...	Response Headers					
notifications/	Allow:	GET, HEAD, OPTIONS				
follow_status/?user_id=...	Content-Encoding:	gzip				
follow_status/?user_id=...	Content-Language:	ko				
follow_status/?user_id=...	Content-Length:	1062				
follow_status/?user_id=...	Content-Type:	application/json				
follow_status/?user_id=...	Date:	Tue, 09 Jan 2024 15:58:13 GMT				
follow_status/?user_id=...	Referrer-Policy:	same-origin				
follow_status/?user_id=...	Server:	Caddy				
follow_status/?user_id=...	Server:	uvicorn				
follow_status/?user_id=...	Set-Cookie:	sessionid=0pqp7k8qkpmoh7c98pe8ky7vd1sxbpdq; Domain=.dreamhack.io; expires=Tue, 16 Jan 2024 15:58:13 GMT; HttpOnly; Max-Age=604800; Path=/; SameSite=Lax				

- 세션 하이재킹(Session Hijacking): 공격자가 쿠키를 훔쳐서 이용자의 인증 상태를 훔치는 것

함께 실습: Cookie

- 인덱스 페이지 코드

```
@app.route('/') # 인덱스 페이지
def index():
    username = request.cookies.get('username', None) # 요청
    의 쿠키에서 이용자 정보 불러옴
    if username:
        return render_template('index.html', text=f'Hello
        {username}, {"flag is " + FLAG if username == "admin" else
        "you are not admin"}') # 쿠키의 username이 admin인 경우
        return render_template('index.html')
```

- 로그인 페이지 코드

- GET : username과 password를 입력할 로그인 페이지 제공
- POST : 입력 값을 users 변수값과 비교

```
@app.route('/login', methods=['GET', 'POST']) # login 페이지
라우팅, GET/POST 메소드로 접근 가능
def login():
    if request.method == 'GET': # GET 메소드로 요청 시
        return render_template('login.html') # login.html
        페이지 출력
    elif request.method == 'POST': # POST 메소드로 요청 시
        username = request.form.get('username') # 이용자가
        전송한 username 입력값을 가져옴
        password = request.form.get('password') # 이용자가
        전송한 password 입력값을 가져옴
        try:
            pw = users[username] # users 변수에서 이용자가 전
            송한 username이 존재하는지 확인
        except:
            return '<script>alert("not found user");histor
            y.go(-1);</script>' # 존재하지 않는 username인 경우 경고 출력
            if pw == password: # password 체크
                resp = make_response(redirect(url_for('index')))
            ) # index 페이지로 이동하는 응답 생성
                resp.set_cookie('username', username) # usernam
            e 쿠키 설정
                return resp
            return '<script>alert("wrong password");history.go
            (-1);</script>' # password가 동일하지 않은 경우
```

- users 변수 선언
 - 손님 계정은 guest, 관리자 계정은 FLAG로 pw가 설정되어 있음

```
try:
    FLAG = open('./flag.txt', 'r').read() # flag.txt 파일로
    부터 FLAG 데이터를 가져옴.
except:
```

```
FLAG = '【**FLAG**】'
users = {
    'guest': 'guest',
    'admin': FLAG # FLAG 데이터를 패스워드로 선언
}
```

- 취약점 분석
 - 요청에 포함된 쿠키에 의해, 이용자의 정보를 나타내는 username이 결정되어 임의로 이용자를 조작하여 로그인할 수 있음.
- 익스플로잇
 - 쿠키에 username = admin 추가

혼자 실습: Session

- 위 문제와 달리 cookie에서 username이 아닌 `sessionid` 를 불러오고, 이것을 key로 하여 `session_storage` 에 접근, 그 값이 admin일 경우 로그인에 성공해 flag를 얻을 수 있음

```
# this is our session storage
session_storage = {
}

@app.route('/')
def index():
    session_id = request.cookies.get('sessionid', None)
    try:
        # get username from session_storage
        username = session_storage[session_id]
    except KeyError:
        return render_template('index.html')

    return render_template('index.html', text=f'Hello {user
```

```
name}, {"flag is " + FLAG if username == "admin" else "you are not
```

- 코드에서 session_storage가 비어 있었고, Application을 보니 Session storage 탭이 있길래 내가 임의로 cookie와 storage에 값을 추가하면 된다고 생각 → 근데 안 됨



- 코드 밑으로 내리니 새로운 코드가 있었음
- 랜덤으로 sessionid를 생성하고 스토리지에 저장하는 코드였는데, print 함수가 있는 걸 보니 어딘가에 출력되는 것 같은데 어딘지 모름
- console에 print(session_storage)를 쳐보기도 했는데 여기가 아닌 것 같음
- 그래서 힌트를 좀 보니까 Flask 문법의 `@app.route` 를 이해해야 하는 것 같았음.

```
@app.route('/admin')
def admin():
    # developer's note: review below commented code and uncomment it (TODO)

    #session_id = request.cookies.get('sessionid', None)
    #username = session_storage[session_id]
    #if username != 'admin':
    #    return render_template('index.html')

    return session_storage

if __name__ == '__main__':
    import os
    # create admin sessionid and save it to our storage
    # and also you cannot reveal admin's sessionid by brute forcing!!! haha
    session_storage[os.urandom(32).hex()] = 'admin'
```

```
print(session_storage)
app.run(host='0.0.0.0', port=8000)
```

Flask에서 `@app.route()` 의 의미

- 첫 번째 매개변수는 경로(URL), 즉 위 코드에서 `/admin` 이 들어가는 새로운 페이지가 있다는 뜻
- `@app` 해당 루트에 방문할 때마다 밑의 함수를 실행하라고 함.

host3.dreamhack.games:17156/admin

로그인 페이지를 보니 '/login'으로 끝나길래 주소를 바꾸어 들어가 봄

```
{ "4e9a3e9246c80d46e338911ee6ce7e4e36077dac9056d7345055a3a7a54c837": "user", "551b51be5d12e06e4d02688b05b58599a0b59291623a54179aea8616ba56bca8": "guest", "82e551b00a7485ae506c466f029e7319edc7c1459e454bac1141e5101d137951": "user", "889dcef7e4a8375ba12e458a1c326eb64690e69a3d963964b23135b594880624": "admin" }
```

admin의 세션ID가 나왔다!

근데 애를 cookie에 추가해도 변화가 없었는데, 알고보니 다른 계정으로 로그인 한 상태에서 sessionid를 바꿔치기 하면 되는 것이었음.

Session
Home

Welcome !
Hello admin,

DevTools is now available in Korean!
Always match Chrome's language
Switch DevTools to Korean
Don't show again

Elements
Console
Sources
Network
Performance
Memory
Application
Security
Light

Application
Manifest
Service workers
Storage

Filter

Name	Value	Do...	Path	Exp...	Size	H
sessionid	889dcef7e4a8375ba12e458a1c326...	host...	/	Ses...	73	
username	admin	host...	/	Ses...	13	

정리

1. 세션 과정 이해
2. Flask 문법 이해 → 새로운 페이지 발견
3. 로그인 상태에서 sessionid 바꿔치기

(session storage 탭은 관련 없는 것이었음)

Same Origin Policy(SOP)

: 동일 출처 정책

탄생 배경

- 이용자의 인증 정보가 담긴 쿠키를 브라우저 내부에 저장하고, 이용자가 웹에 접속하면 HTTP 요청에 쿠키를 포함시키는 것이 웹의 특징.
- 직접 접속이 아닌, 웹 리소스를 통한 접속도 마찬가지로 HTTP 요청에 쿠키를 포함시킴.
- 따라서 공격자의 페이지가 대상 사이트에 HTTP 요청을 보내고 응답 정보를 받는 코드를 실행할 수 있음. → 보안 위협 가능성

Origin(정보 출처) 구별 방법

프로토콜(Protocol, Scheme), 포트, 호스트로 구성

구성 요소가 **모두 일치**해야 동일한 오리진

여기서 오리진은 개발자 도구의 콘솔에 `location` 을 입력하면 볼 수 있음

URL	결과	이유
https://same-origin.com/frame.html	Same Origin	Path만 다름
http ://same-origin.com/frame.html	Cross Origin	Scheme이 다름
https:// cross .same-origin.com/frame.html	Cross Origin	Host가 다름
https://same-origin.com: 1234 /	Cross Origin	Port가 다름

Same Origin 실습

`window.open` : 새로운 창 띄우기

`object.location.href` : 객체가 가리키는 URL 주소를 읽어오는 코드

- Same origin 일 경우

```
sameNewWindow = window.open('https://dreamhack.io/lecture');  
console.log(sameNewWindow.location.href);  
결과: https://dreamhack.io/lecture
```

- Cross origin일 경우

```
crossNewWindow = window.open('https://theori.io');  
console.log(crossNewWindow.location.href);  
결과: 오류 발생
```

위와 같이 외부 출처에서 불러온 데이터를 읽을 때는 오류가 발생하지만
데이터를 쓰는 것은 가능

```
crossNewWindow = window.open('https://theori.io');  
crossNewWindow.location.href = "https://dreamhack.io";
```

교차 출처 리소스 공유(CORS, Cross Origin Resource Sharing)

: 브라우저에서 외부 출처에 대한 접근 제한을 완화하여 오리진이 다른 사이트끼리 리소스를 공유할 수 있게 하는 방법

이때 HTTP 헤더를 이용함

- 발신측의 웹 리소스 요청 코드

- XMLHttpRequest() 는 서버와 상호작용할 때 사용하는 객체로, 페이지를 새로고침할 필요 없이 URL에서 데이터를 가져올 수 있음 (xml 외에도 가능)

```
/*
    XMLHttpRequest 객체를 생성합니다.
    XMLHttpRequest는 웹 브라우저와 웹 서버 간에 데이터 전송을
    도와주는 객체 입니다. 이를 통해 HTTP 요청을 보낼 수 있습니다.
*/
xhr = new XMLHttpRequest();
/* https://theori.io/whoami 페이지에 POST 요청을 보내도록 합니다. */
xhr.open('POST', 'https://theori.io/whoami');
/* HTTP 요청을 보낼 때, 쿠키 정보도 함께 사용하도록 해줍니다. */
xhr.withCredentials = true;
/* HTTP Body를 JSON 형태로 보낼 것이라고 수신측에 알려줍니다. */
xhr.setRequestHeader('Content-Type', 'application/json');
/* xhr 객체를 통해 HTTP 요청을 실행합니다. */
xhr.send("{\"data\":\"WhoAmI\"}");
```

- 발신측의 HTTP 요청

- 위 코드에서는 POST 요청을 보냈으나 실제로 OPTIONS 메소드로 요청이 보내짐
- **CORS preflight**: 수신측에 웹 리소스를 요청해도 되는지 질의하는 것

```
OPTIONS /whoami HTTP/1.1
Host: teori.io
Connection: keep-alive
Access-Control-Request-Method: POST
Access-Control-Request-Headers: content-type
Origin: https://dreamhack.io
Accept: */*
Referer: https://dreamhack.io/
```

- 서버 응답 결과

HTTP/1.1 200 OK

Access-Control-Allow-Origin: https://dreamhack.io

Access-Control-Allow-Methods: POST, GET, OPTIONS

Access-Control-Allow-Credentials: true

Access-Control-Allow-Headers: Content-Type

- 응답 결과에 대한 설명

Header	설명
Access-Control-Allow-Origin	헤더 값에 해당하는 Origin에서 들어오는 요청만 처리합니다.
Access-Control-Allow-Methods	헤더 값에 해당하는 메소드의 요청만 처리합니다.
Access-Control-Allow-Credentials	쿠키 사용 여부를 판단합니다. 예시의 경우 쿠키의 사용을 허용합니다.
Access-Control-Allow-Headers	헤더 값에 해당하는 헤더의 사용 가능 여부를 나타냅니다.

> 수신측의 응답이 요청과 상응하는지 확인, 이후 원래 보내고자 했던 POST 요청을 보냄

JSON with Padding (JSONP)

이미지, javascript, CSS 등의 리소스는 SOP 제한 없이 외부 출처 접근 허용

→ 이를 이용해 `<script>` 태그로 Cross origin의 데이터 불러오는 것

(`<script>` : 데이터와 실행 가능한 코드를 문서에 포함할 때 사용)

- 요청하는 코드

- <script> 태그 안의 데이터는 js 코드로 인식하므로 **callback** 함수 활용

```
<script>
/* myCallback이라는 콜백 함수를 지정합니다. */
function myCallback(data){
    /* 전달받은 인자에서 id를 콘솔에 출력합니다.*/
    console.log(data.id)
}
</script>
<!--
https://theori.io의 스크립트를 로드하는 HTML 코드입니다.
단, callback이라는 이름의 파라미터를 myCallback으로 지정함으로써
수신측에게 myCallback 함수를 사용해 수신받겠다고 알립니다.
-->
<script src='http://theori.io/whoami?callback=myCallback'>
</script>
```

- 수신측 응답 코드

```
/*
수신측은 myCallback 이라는 함수를 통해 요청측에 데이터를 전달합니
다.
전달할 데이터는 현재 theori.io에서 클라이언트가 사용 중인 계정 정보
인
{'id': 'dreamhack'} 입니다.
*/
myCallback({'id': 'dreamhack'});
```

CORS가 생기기 전 사용되던 방법으로, 현재는 거의 쓰이지 않음

문제

1. session

- 앞선 session 문제와 비슷하지만, session이 따로 출력되는 페이지가 없고 sessionid의 길이가 1바이트로 줄었음
- 16진수이므로 두 글자 → 직접 브루트포스로 해결하는 것

```
if __name__ == '__main__':  
    import os  
    session_storage[os.urandom(1).hex()] = 'admin'  
    print(session_storage)  
    app.run(host='0.0.0.0', port=8000)
```

해결방법

- burp suite의 Intruder 기능을 이용 → 두 자리수의 16진수를 모두 대입해서 응답 확인하기

burp suite Intruder 사용법

- 로그인 후 GET 요청을 Intruder로 보낸다.
- 값을 대입해 줄 부분을 드래그 한 후 오른쪽 Add를 클릭해 추가



- Payload로 이동, 16진수는 다음과 같이 입력
- 숫자로도 입력할 수 있는데 헛갈려서 안 해봄



Payload sets

You can define one or more payload sets. The number of payload sets depends on the a payload type can be customized in different ways.

Payload set: 1 Payload count: 256

Payload type: Brute forcer Request count: 256



Payload settings [Brute forcer]

This payload type generates payloads of specified lengths that contain all permutations of

Character set: abcdef1234567890

Min length: 2

Max length: 2

- Settings의 Grep 기능을 활용하면, 특정 단어가 나왔을 때 표시해주도록 설정 가능
- 나는 flag를 추가했다.



Grep - Match



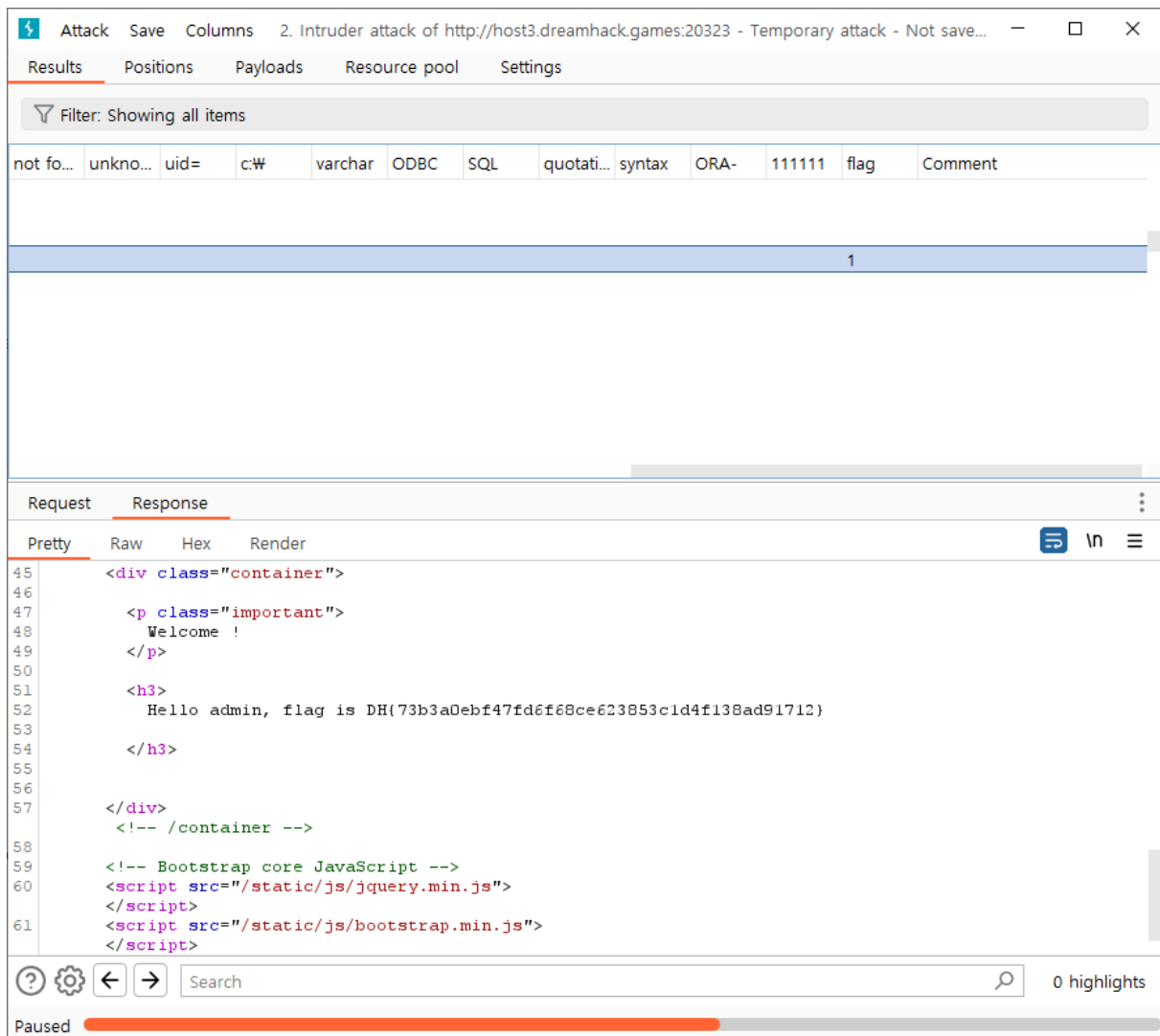
These settings can be used to flag result items containing specified expressions.

☒ Flag result items with responses matching these expressions:

Paste	exception
Load ...	illegal
Remove	invalid
Clear	fail
	stack
	access
	directory
Add	flag

- Start attack을 누르면 공격을 시작함

- length나 앞서 설정한 flag를 보면 어떤 게 flag 응답인지 알 수 있다.
- 우클릭으로 브라우저에서도 볼 수 있음



- 파이썬이나 JS로 직접 코드 짤 수 있으면 좋을텐데...^

2. Cookies

- Home에서는 쿠키값이 -1, snickerdoodle을 입력하면 0으로 바뀜
- -1에서 0이 됐으니까 다른 것들도 양수 쪽으로 간다고 생각하는 게 상식적인데 나는 무지성으로 -2 넣어보고 어? 안되네 하고 잘못된 접근이라고 생각했다... 😅

Name	Value
name	-1

That is a cookie! Not very special though...



I love snickerdoodle cookies!

© PicoCTF

- 1부터 넣어보면 다양한 종류의 쿠키가 뜬다.
- description이 'Try to figure out the **best one**.' 이니까 이 중 답이 있을 것

3. Power Cookie

처음 들어가면 guest로는 접속할 수 없다고 뜨고, isAdmin이라는 쿠키가 0으로 설정되어 있음

1로 바꿔주면 플래그가 뜬다.

Name	Value
isAdmin	1

4. old-01

```
<?php
    include "../../config.php";
    if($_GET['view-source'] == 1){ view_source(); }
    if(!$_COOKIE['user_lv']){ // 쿠키가 없다면
        SetCookie("user_lv","1",time()+86400*30,"/challenge/web
-01/"); // 1로 설정해주기
        echo("<meta http-equiv=refresh content=0>");
    }
?>
<html>
<head>
<title>Challenge 1</title>
</head>
<body bgcolor=black>
<center>
<br><br><br><br><br>
<font color=white>
-----<br>
<?php
    if(!is_numeric($_COOKIE['user_lv'])) $_COOKIE['user_lv']=
1;
    if($_COOKIE['user_lv']>=4) $_COOKIE['user_lv']=1;
    if($_COOKIE['user_lv']>3) solve(1); //내장함수가 아니다! 뭔
가 있어보임
    echo "<br>level : {$_COOKIE['user_lv']}";
?>
<br>
<a href=./?view-source=1>view-source</a>
```

```
</body>
</html>
```

코드 분석하기

서버사이드 스크립트인 php

```
// 쿠키 설정하기
setcookie (쿠키명, 쿠키값, 만료시간, 경로, 도메인, 보안, httponly);
```

```
//0초마다, 즉 계속해서 새로고침 됨
echo("<meta http-equiv=refresh content=5>")
```

```
// 쿠키값 읽기
$_COOKIE[쿠키명]
```

쿠키값이 2, 3일 때만 level : 2, level : 3으로 바뀜

그 외에는 모두 1

해결법

```
<?php
    if(!is_numeric($_COOKIE['user_lv'])) $_COOKIE['user_lv']=
    1;
    if($_COOKIE['user_lv']>=4) $_COOKIE['user_lv']=1;
    if($_COOKIE['user_lv']>3) solve(1); //내장함수가 아니다! 뭔
    가 있어보임
    echo "<br>level : {$_COOKIE['user_lv']}";
?>
```

- solve 함수를 실행시키려면 쿠키값(user_lv)이 3보다 커야 함
- 그러나 4 이상이면 1로 설정됨
- 따라서 3보다 크고 4보다 작은 값을 넣어주었더니 해결됨

